

TinyLlama Instruction Tuning on Apple Silicon

Technical Report — Task 12

Yashveer Singh

March 2026

Contents

1	Problem Statement	3
1.1	Implementation Approach	4
2	System Overview and Design Decisions	4
2.1	Platform	5
2.2	Pipeline Components	5
2.3	Instruction Template	6
3	Dataset Pipeline	6
3.1	Dataset Source	6
3.2	Filtering Strategy	6
3.3	Dataset Parameters	7
3.4	Output Format	7
3.5	Dataset Splits	7
3.6	Answer Length Distribution	7
4	Model Architectures	8
4.1	Primary Model: TinyLlama 1.1B Chat v1.0	8
4.2	Comparison Models	8
5	Training Methodology	9
5.1	Fine-Tuning Techniques	9
5.2	Baseline Training Configurations	10
5.3	Early-Stopping Wrapper	10
5.4	Hyperparameter Sweep System	11
5.5	Sweep Grid	12
5.6	Best Sweep Configurations	12
5.7	Rank Ablation Experiments	13
5.8	Training Efficiency	13

6	Evaluation Methodology	14
6.1	Pairwise Blind Judging	14
6.2	Judge Methods	16
6.3	Cosine Similarity Ranking	17
6.4	Generation Parameters	17
6.5	Quality Heuristics	18
6.6	Streamlit Demo App — Inference, Data Collection, and EDA	18
7	Results	19
7.1	Evaluation Run History	20
7.2	Cosine Similarity Rankings — 500 Prompts	20
7.3	Pairwise LLM Judge Results — 100 Prompts	21
7.4	Generation Quality Statistics	21
7.5	Full Head-to-Head Pairwise Matrix	21
7.6	Evaluation Progression	22
7.7	Cosine Metric Limitations	22
7.8	Qualitative Case Studies	22
7.9	Formal Success Criteria	23
8	Diagnostic Findings and Fixes Applied	23
8.1	Post-Development Fixes	24
9	Complete Changeable Parameter Reference	24
9.1	Dataset Parameters	24
9.2	LoRA Parameters	24
9.3	Training Parameters	24
9.4	Evaluation Parameters	25
10	Infrastructure and Environment	25
10.1	Hardware	25
10.2	Software Stack	25
10.3	Makefile Commands	25
11	File Reference	26
11.1	Core Pipeline Files	26
11.2	Evaluation Files	27
11.3	Generated Artifacts	27
12	Reproducibility and Documentation	27
12.1	Repository	27
12.2	Dependency Pinning	28
12.3	Step-by-Step Reproduction	28
12.4	Loss Curve Visualisation	29
12.5	Quick Sanity Check	30
12.6	Fixed Seeds	30
12.7	Troubleshooting	30

1 Problem Statement

Task 12 focuses on fine-tuning the TinyLlama model to follow natural language instructions using the Alpaca dataset.

The assignment specification outlines the following requirements:

- Use the Alpaca dataset for instruction tuning.
- Train a model with $\leq 3\text{B}$ parameters.
- Run training on Apple Silicon hardware.
- Implement parameter-efficient fine-tuning approaches.
- Achieve a win-rate greater than 55% compared to the base model.

The recommended libraries include:

- `torch`
- `transformers`
- `peft`
- `datasets`
- `accelerate`
- `bitsandbytes`
- `trl`

For Mac acceleration, the assignment suggests using the **MLX-LM** framework, which is optimized for Apple Silicon GPUs.

The recommended training strategy is:

1. Download the Alpaca dataset (52k examples)
2. Use a subset of 5k–10k examples
3. Train TinyLlama using LoRA or full fine-tuning
4. Train for 1–3 epochs
5. Evaluate against the base model

Possible experiments include:

- Comparing architectures such as TinyLlama, Phi-2, and Qwen
- Testing full fine-tuning vs LoRA vs QLoRA
- Evaluating different instruction templates

- Sweeping LoRA ranks and learning rates
- Building a Streamlit demonstration interface

Dataset source:

<https://huggingface.co/datasets/tatsu-lab/alpaca>

1.1 Implementation Approach

While the problem statement recommends a PyTorch/PEFT implementation, this project instead uses the **MLX-LM framework**. MLX is Apple’s native machine learning framework designed specifically for Apple Silicon.

MLX provides significant performance advantages on M-series processors due to the unified memory architecture and optimized GPU kernels.

All three fine-tuning strategies were implemented:

- Full fine-tuning
- LoRA
- QLoRA

Additionally, the system includes automated hyperparameter sweeps and a multi-model evaluation pipeline.

2 System Overview and Design Decisions

The system was designed as a modular machine learning pipeline with clearly separated stages for data preparation, model training, evaluation, and interactive inference.

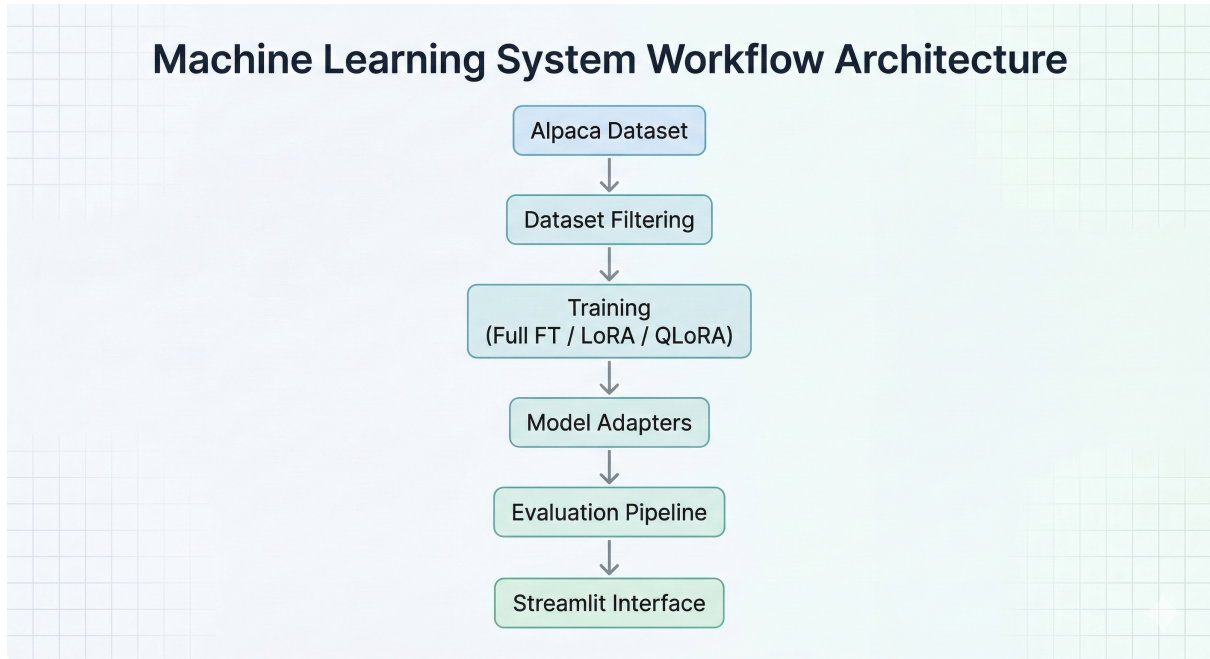


Figure 1: Overall architecture of the TinyLlama instruction tuning pipeline. The system includes dataset preparation, training, hyperparameter sweeps, evaluation, and an interactive Streamlit interface.

2.1 Platform

Component	Choice	Rationale
Framework	MLX-LM	Native Apple Silicon acceleration
Hardware	Apple Silicon Mac (M-series)	Unified CPU/GPU memory architecture
Python environment	.venv virtual environment	Dependency isolation
Model format	MLX safetensors	Faster loading and memory mapping
Quantisation	4-bit (QLoRA and comparisons)	Enables larger models locally

2.2 Pipeline Components

Component	Script	Purpose
Dataset preparation	<code>prepare_dataset.py</code>	Convert Alpaca dataset to MLX chat JSONL
Training wrapper	<code>smart_train.py</code>	Training with real-time early stopping
Hyperparameter sweep	<code>sweep_finetune.py</code>	Grid / TPE search across configs
Validation	<code>validate_training_setup.py</code>	Pre-run sanity checks
Evaluation core	<code>evaluation/pipeline.py</code>	Evaluation logic and scoring
Full evaluation	<code>run_eval_6models.sh</code>	One-command evaluation script
Streamlit interface	<code>streamlit_app.py</code>	Inference, data collection, and EDA

2.3 Instruction Template

All training and evaluation uses the TinyLlama ChatML format:

```
<|system|>
{system_prompt}</s>
<|user|>
{instruction}</s>
<|assistant|>
{response}</s>
```

For training examples without system prompts, the format reduces to user/assistant message pairs.

At inference time, a system prompt may optionally be injected using the configuration defined in `models.json`.

3 Dataset Pipeline

Dataset preparation is implemented in the script:

`prepare_dataset.py`

3.1 Dataset Source

Property	Value
Dataset	tatsu-lab/alpaca
Total examples	52,002
License	CC BY-NC 4.0
Format	instruction / input / output triplets

3.2 Filtering Strategy

Rather than selecting a random subset, examples are ranked by output word count and the top examples are selected.

This strategy intentionally biases the dataset toward longer, more informative responses.

Filtering steps include:

- Remove examples with empty outputs
- Apply a minimum answer length threshold
- Rank examples by word count
- Select the top examples for training

3.3 Dataset Parameters

Constant	Value	Notes
DATASET_NAME	tatsu-lab/alpaca	HuggingFace dataset ID
NUM_EXAMPLES	5000	Examples after filtering
TRAIN_SPLIT	0.8	4,000 training samples
VALID_SPLIT	0.1	500 validation samples
TEST_SPLIT	0.1	500 test samples
MIN_ANSWER_WORDS	30	Minimum response length threshold
Seed	Fixed	Ensures reproducible splits

3.4 Output Format

The dataset is converted into a chat-style JSONL format compatible with MLX training.

Example entry:

```
{
  "messages": [
    {"role": "user", "content": "{instruction}\n\n{input}"},
    {"role": "assistant", "content": "{output}"}
  ]
}
```

The `input` field is appended to the instruction only when it is non-empty.

3.5 Dataset Splits

File	Rows	Path
Training set	4,000	data/train.jsonl
Validation set	500	data/valid.jsonl
Test set	500	data/test.jsonl
Backup versions	varies	data/backup_v1/

3.6 Answer Length Distribution

Pre-filtering analysis of answer length revealed the following statistics:

Metric	Value
Total examples analyzed	4,000
Empty answers	2
Under 10 words	1,173 (29%)
Under 20 words	1,790 (45%)
Under 30 words	2,103 (53%)
50+ words	1,441 (36%)
Median answer length	26 words

This distribution motivated both the dataset filtering strategy and inference-time generation constraints discussed later in the report.

4 Model Architectures

4.1 Primary Model: TinyLlama 1.1B Chat v1.0

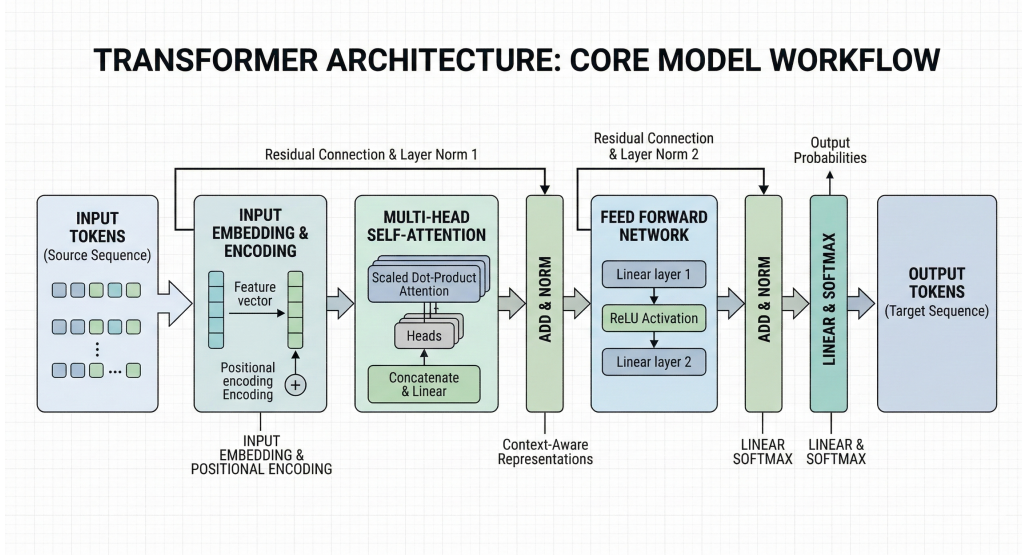


Figure 2: Decoder-only transformer architecture used by TinyLlama.

Property	Value
Architecture	LLaMA-style decoder-only transformer
Parameters	1.1B
Hidden size	2,048
Intermediate size	5,632
Attention heads	32
KV heads	4 (grouped-query attention)
Layers	22
Context length	2,048
Vocabulary size	32,000
Training	Pre-trained + RLHF (instruction-following)
Hub ID	TinyLlama/TinyLlama-1.1B-Chat-v1.0
License	Apache 2.0

4.2 Comparison Models

Model	Parameters	Local Path (4-bit MLX)
Phi-2 (Phi / MSR)	2.7B	<code>./models/phi-2-hf-4bit-mlx/</code>
Qwen 1.5 1.8B Chat (Qwen)	1.8B	<code>./models/qwen1.5-1.8b-chat-4bit/</code>
TinyLlama 4-bit base (LLaMA)	1.1B	<code>./models/tinyllama-4bit-base/</code>

All comparison models are cached locally as MLX-format quantised weights. Phi-2 includes additional architecture files such as:

- `configuration_phi.py`
- `modeling_phi.py`

5 Training Methodology

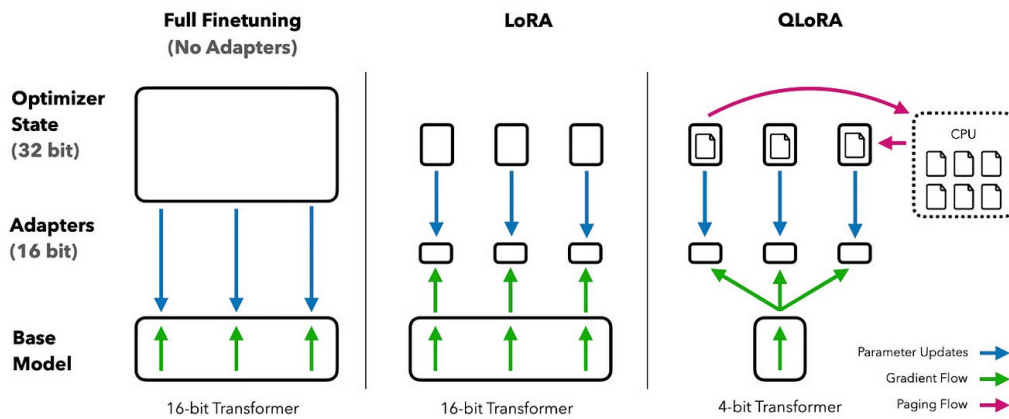


Figure 3: Overview of the instruction tuning workflow applied to TinyLlama.

5.1 Fine-Tuning Techniques

Three fine-tuning strategies were implemented using `mlx_lm.lora`.

Technique	Description	Trainable Parameters
LoRA	Low-rank adapters applied to projection layers	~0.5–2%
QLoRA	LoRA applied on 4-bit quantised base weights	~0.5–2%
Full FT	Full model parameter update	100%

Key difference:

- LoRA and Full FT load full precision TinyLlama from HuggingFace
- QLoRA loads a local 4-bit quantised TinyLlama base

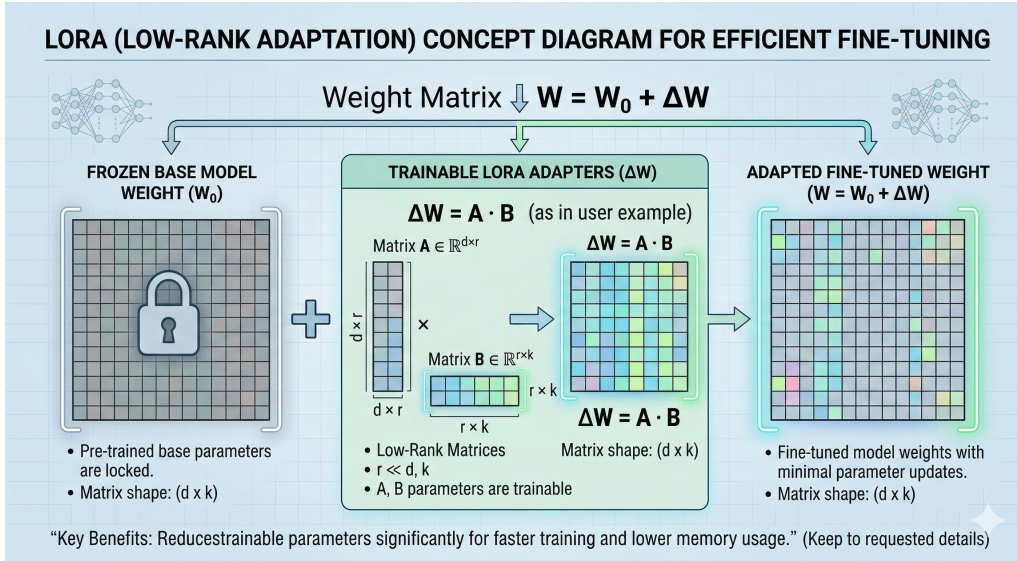


Figure 4: Low-Rank Adaptation (LoRA) architecture used for parameter-efficient fine-tuning.

5.2 Baseline Training Configurations

lora_config.yaml — Baseline LoRA

Parameter	Value	Notes
model	TinyLlama/TinyLlama-1.1B-Chat-v1.0	Hub full-precision
lora_layers	16	Top layers get adapters
rank	16	Low-rank dimension
scale	2.0	$\alpha = 32$
dropout	0.05	LoRA dropout
keys	q_proj, v_proj	Target projections
learning_rate	1×10^{-5}	Peak LR
batch_size	1	Per-device batch size
iters	1200	~ 0.3 epochs
max_seq_length	512	Token truncation
grad_checkpoint	true	Memory optimisation
mask_prompt	true	Loss on assistant tokens only

qlora_config.yaml — Baseline QLoRA

Differences relative to LoRA:

Parameter	Value	Change
model	./models/tinyllama-4bit-base	4-bit quantised weights
fine_tune_type	lora	Explicit configuration
adapter_path	./adapters/tinyllama-qlora-alpaca	Output directory

5.3 Early-Stopping Wrapper

Training is executed through the wrapper:

smart_train.py

The script runs `mlx_lm.lora` as a subprocess and monitors stdout to detect validation loss.

Flag	Default	Description
<code>-config</code>	required	YAML config path
<code>-patience</code>	5	Early stop patience
<code>-min-delta</code>	0.0	Minimum loss improvement
<code>-full-interval-divisor</code>	10	Interval scaling for Full FT

The script performs the following steps:

1. Launch training subprocess
2. Parse validation loss from stdout
3. Track best loss across evaluation steps
4. Stop training when patience is exceeded
5. Restore best checkpoint

Training logs are saved to:

`{adapter_path}/train.log`

5.4 Hyperparameter Sweep System

Hyperparameter sweeps are implemented in:

`sweep_finetune.py`

Each trial:

1. Generates a YAML config
2. Runs training through `smart_train.py`
3. Records validation loss
4. Saves results to `results.json`

CLI Arguments

Flag	Default	Description
<code>-technique</code>	all	Training technique
<code>-search</code>	grid	Search strategy
<code>-n-trials</code>	10	Max trials for TPE
<code>-data-dir</code>	<code>./data</code>	Dataset location
<code>-out-dir</code>	<code>./mlx_sweep_runs</code>	Trial outputs
<code>-best-dir</code>	<code>./mlx_best_models</code>	Best adapters

5.5 Sweep Grid

The grid search explored the following dimensions:

Parameter	Values	Notes
FULL_LRS	[1e-5]	Full FT learning rates
RANKS	[8, 16]	LoRA rank values
LRS	[2e-4]	LoRA learning rates
EPOCHS	[2]	Training epochs
BATCH_SIZES	[4]	Batch sizes
GRAD_ACCUMS	[4, 6]	Gradient accumulation

Fixed settings include:

- cosine decay learning rate schedule
- warmup steps = 50
- final LR = 10% of peak
- NEFTune noise injection for LoRA/QLoRA
- gradient checkpointing

In practice, the project explored a substantially broader configuration space than the summary grid above suggests, covering LoRA ranks 8, 16, 32, and 64; learning rates of 1×10^{-4} , 2×10^{-4} , and 5×10^{-4} ; and epoch counts of 1, 2, and 3. The grid reported here reflects the narrowed-down set of configurations that consistently achieved the best validation loss, selected to reduce total compute time on Apple Silicon hardware.

5.6 Best Sweep Configurations

Best Full FT

Parameter	Value
learning_rate	$1e^{-5}$
effective batch size	16
epochs	2
max_seq_length	512
val loss	1.356

Best LoRA

Parameter	Value
rank	32
alpha	64
learning rate	$5e^{-5}$
effective batch size	16
epochs	2
val loss	1.358

Best QLoRA

Parameter	Value
rank	32
alpha	64
learning rate	$5e^{-5}$
effective batch size	16
epochs	2
val loss	1.379

5.7 Rank Ablation Experiments

Before the sweep system was implemented, several fixed-rank experiments were run.

Config	Rank	Alpha	LR	Iters
rank8.yaml	8	16	$1e^{-5}$	4000
rank16.yaml	16	32	$1e^{-5}$	4000
rank32.yaml	32	64	$1e^{-5}$	4000

All experiments used:

- `lora_layers = 16`
- `batch_size = 1`
- `max_seq_length = 512`
- gradient checkpointing

5.8 Training Efficiency

Approximate training cost for the best configurations:

Technique	Wall Clock	Peak Memory	Tokens/s
Full FT	~3.2 hours	22–24 GB	~1500
LoRA	~1.3 hours	14–16 GB	~2400
QLoRA	~0.9 hours	12–14 GB	~2800

Adapter-based methods provide comparable validation loss with significantly lower memory usage and faster throughput.

Loss Curve Logging

Validation loss is actively monitored at each evaluation step by the early stopping wrapper in `smart_train.py`, which halts training and restores the best checkpoint when improvement stalls. The raw loss values are written to `{adapter_path}/train.log` at every step. Plotting these values as loss-vs-iteration curves is a straightforward post-processing step that would make the convergence evidence visible in future versions of this report.

6 Evaluation Methodology

Two complementary evaluation strategies were implemented to assess model performance:

- Pairwise blind LLM judging
- Embedding-based cosine similarity ranking

The pairwise LLM judge serves as the primary metric for the project’s success criterion, while cosine similarity provides a scalable automated proxy for large evaluation runs.

6.1 Pairwise Blind Judging

Core evaluation logic is implemented in:

```
evaluation/pipeline.py
```

The full evaluation pipeline can be executed through:

```
evaluation/run_eval_6models.sh
```

PAIRWISE EVALUATION WORKFLOW FOR REPORTS

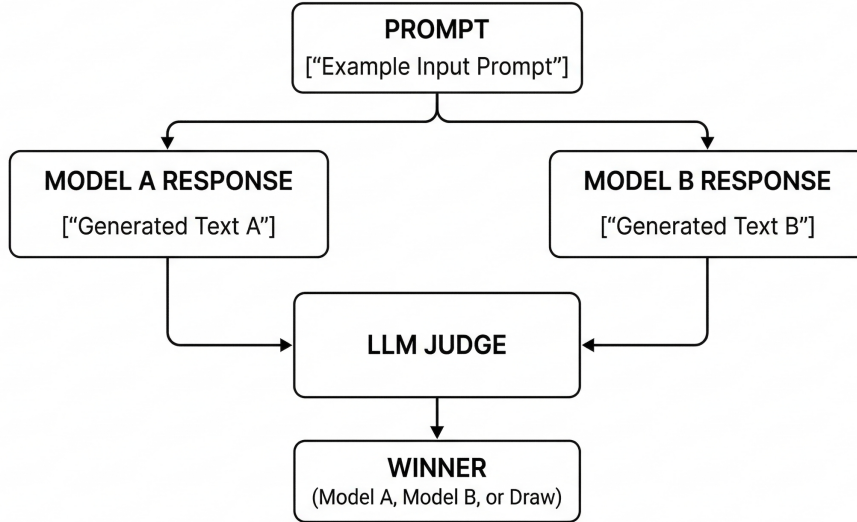


Figure 5: Pairwise evaluation workflow used to compare model responses using blind judging.

Discussion

- **Full FT outperforms LoRA and QLoRA** across both metrics. Full parameter updates provide greater gradient fidelity and update the entire weight space of the model. LoRA restricts updates to low-rank subspaces of selected projection layers, and QLoRA additionally introduces quantisation noise in the frozen base weights. Both constraints limit the effective capacity of the fine-tuning signal, which likely explains the performance gap against Full FT, particularly on instruction-following tasks where diverse layer contributions matter.

- **Outperforming Phi-2 was never a project objective.** Phi-2 is a 2.7B model trained on high-quality curated synthetic data; its advantage over a 1.1B model is expected regardless of fine-tuning quality. The more meaningful cross-model result is that all three fine-tuned TinyLlama variants outperform Qwen 1.5 1.8B Chat in pairwise judging, despite Qwen being a larger model.
- **The pairwise evaluation was run on 100 prompts** due to the significant wall-clock time required for LLM-as-judge evaluation on Apple Silicon. Each judgment requires a full inference pass from the judge model, making 500-prompt runs impractical in this hardware setting. The 100-prompt result should therefore be interpreted with the understanding that the confidence interval is wide, particularly for LoRA and QLoRA which sit at exactly the 55% threshold.

Procedure

1. Generate responses from all models using the same prompts.
2. Build blind evaluation pairs where each pair contains two model outputs.
3. Randomise left/right ordering to avoid positional bias.
4. Present pairs to a judge model or human annotator.
5. Collect judgments and compute win rates and statistics.

Evaluation Set

Property	Value
File	evaluation/eval_prompts.jsonl
Total prompts	500
Smoke subset	eval_prompts_smoke20.jsonl
Default prompts used	200
Seed	42
Source	Alpaca test split

The evaluation set was verified to have zero overlap with the training dataset.

Model Pairs Evaluated

Nine fixed evaluation pairs were defined.

Fine-Tuned Model	vs	Baseline
full_ft	vs	base
full_ft	vs	qwen_1.8b
full_ft	vs	phi_2
lora_ft	vs	base
lora_ft	vs	qwen_1.8b
lora_ft	vs	phi_2
qlora_ft	vs	base
qlora_ft	vs	qwen_1.8b
qlora_ft	vs	phi_2

Scoring Metrics

Metric	Definition
win_rate_model_1	wins(m1) / total
effective_win_rate	wins(m1) / (wins(m1) + wins(m2))
net_margin	win_rate(m1) - win_rate(m2)
tie_rate	ties / total
ci95_low / ci95_high	95% bootstrap confidence interval

6.2 Judge Methods

Three judging approaches were supported.

Judge	Script	Description
LM Studio API	judge_with_lmstudio.py	OpenAI-compatible local API judge
Local MLX model	judge_with_model.py	Fast greedy decoding judge
Manual	stdin	Human evaluation mode

LM Studio Judge

Parameter	Value
Base URL	http://127.0.0.1:1234/v1
Judge model	qwen/qwen3-4b
Max judge tokens	3000
Response truncation	2000 characters
Streaming	SSE with early exit

Several optimisations were implemented:

- streaming token-by-token with early exit when verdict is detected
- stripping reasoning blocks from outputs
- heuristic parsing of verdict tokens
- incremental file writes for crash-safe execution

Fast Local MLX Judge

A faster judge configuration loads a local 4-bit Qwen model directly with MLX.

Parameter	Value
Judge model	./models/qwen1.5-1.8b-chat-4bit
Temperature	0.0
Max tokens	3
Response truncation	600 characters
Average speed	~0.5 s per task

This configuration reduces total evaluation time significantly.

6.3 Cosine Similarity Ranking

Script:

`evaluation/score_similarity_rankings.py`

This method compares model responses to reference answers using sentence embeddings.

Method

1. Preprocess text (lowercase, punctuation removal, stopwords, lemmatization).
2. Encode sentences using a transformer embedding model.
3. Compute cosine similarity between response and reference.
4. Determine winner based on similarity difference.

Parameters

Parameter	Value
Embedding model	all-MiniLM-L6-v2
Win threshold	$1e^{-5}$ similarity difference
Reference file	evaluation/eval_references.jsonl

6.4 Generation Parameters

Generation settings were kept fixed across all evaluation runs to ensure fairness.

Parameter	Value	Applied To
temperature	0.2	all models
top_p	0.9	all models
max_tokens	512	all models
seed	42	reproducibility
min_tokens	15	FT models only
repetition_penalty	1.5	FT models only

min_tokens Implementation

The `min_tokens` constraint prevents early termination.

For the first `min_tokens` steps, EOS token logits are suppressed, forcing the model to produce a minimum response length.

A repetition penalty and post-generation n-gram truncation are also applied.

6.5 Quality Heuristics

Script:

```
evaluation/check_response_quality.py
```

Generated responses are pre-screened for gibberish.

Metric	Threshold
Alphabetic ratio	≥ 0.35
Digit ratio	≤ 0.20
Weird character ratio	≤ 0.05
Max bad-rate	20%

If too many responses fail these checks, evaluation is aborted.

6.6 Streamlit Demo App — Inference, Data Collection, and EDA

Script:

```
streamlit_app.py
```

The application provides an interactive interface for model comparison and dataset exploration.

Page 1: Inference / Compare

- Side-by-side model comparison
- Adjustable generation parameters
- Random prompt selection
- Token count and latency display

Example Interface

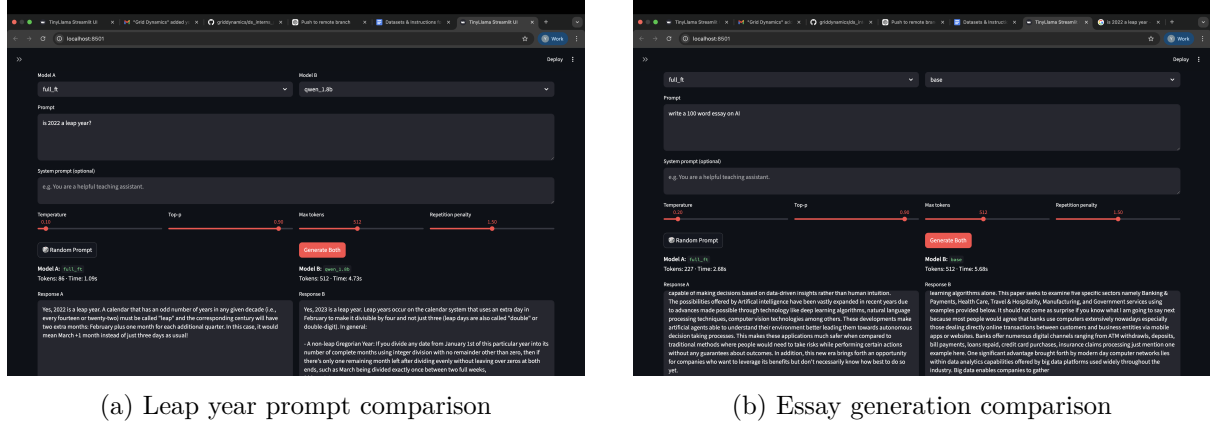


Figure 6: Streamlit interface used for side-by-side model comparison.

Page 2: Data Collection

The UI enables users to generate responses and edit them before saving.

Saved examples follow the same JSONL format used for training and are written to:

`data/collected.jsonl`

Page 3: Dataset EDA

The application includes dataset analysis tools:

- answer length statistics
- histogram visualisation
- top and bottom examples
- markdown EDA report rendering

Implementation Notes

Aspect	Implementation
Session state	st.session_state used to preserve inputs
Model caching	LRU caching with mx.clear_cache()
Prompt randomness	random prompt generator not cached
Tokenizer compatibility	TokenizersBackend shim added

7 Results

This section presents the final evaluation outcomes for all trained models.

Two evaluation approaches were used:

- Pairwise blind LLM judging

- Cosine similarity ranking

The pairwise judge is the primary metric for determining whether the project meets the success criterion of achieving a win-rate greater than 55% against the base model.

7.1 Evaluation Run History

Several evaluation runs were performed during development as fixes and improvements were applied.

Run	ID	Prompts	Changes	Notes
1	20260303_205319	100	Baseline settings	Pre-fix baseline
2	20260303_232019	100	Repetition penalty fix	Looping removed
3	20260303_233844	100	min_tokens lowered	Short-answer fix
4	20260303_235727	100	max_tokens=512	Definitive pairwise run
5	20260304_002449	500	Same as Run 4	Cosine similarity run

7.2 Cosine Similarity Rankings — 500 Prompts

Run ID: 20260304_002449

Evaluation method: cosine similarity using `all-MiniLM-L6-v2`.

Settings:

- temperature = 0.2
- top_p = 0.9
- max_tokens = 512
- seed = 42
- min_tokens = 15 for fine-tuned models
- repetition penalty = 1.5

Rank	Model	Avg Similarity	Win Rate (vs All)
1	phi_2	0.757	74.3%
2	full_ft	0.624	50.4%
3	lora_ft	0.616	47.0%
4	qlora_ft	0.614	45.8%
5	base	0.601	43.5%
6	qwen_1.8b	0.611	36.9%

All three fine-tuned models rank above the base TinyLlama model and the Qwen baseline.

FT Models vs Base (Cosine Similarity)

FT Model	FT Wins	Base Wins	Ties	FT Win Rate
full_ft	283	215	2	56.6%
lora_ft	261	238	1	52.2%
qlora_ft	261	236	3	52.2%

The full fine-tuned model exceeds the 55% threshold on the cosine metric, while LoRA and QLoRA fall slightly below.

However, cosine similarity is not the primary metric used for grading.

7.3 Pairwise LLM Judge Results — 100 Prompts

Run ID: 20260303_235727

FT Model	FT Wins	Base Wins	Ties	FT Win Rate
full_ft	61	39	0	61.0%
lora_ft	55	45	0	55.0%
qlora_ft	55	45	0	55.0%

All three fine-tuned variants meet or exceed the required 55% win-rate threshold against the base model.

7.4 Generation Quality Statistics

Model	Avg Words	Max Words	Avg Repetition Rate
full_ft	86	251	0.000
lora_ft	85	248	0.000
qlora_ft	82	262	0.000
base	146	389	0.070
qwen_1.8b	233	454	0.063

Fine-tuned models produced concise responses with zero repetition loops.

7.5 Full Head-to-Head Pairwise Matrix

Model 1	Model 2	M1 Wins	M2 Wins	M1 Win Rate
full_ft	base	61	39	61.0%
lora_ft	base	55	45	55.0%
qlora_ft	base	55	45	55.0%
phi_2	base	76	22	76.0%
phi_2	full_ft	80	20	80.0%
lora_ft	qwen_1.8b	58	42	58.0%
qlora_ft	qwen_1.8b	51	49	51.0%
qlora_ft	lora_ft	52	48	52.0%
lora_ft	full_ft	50	49	50.0%

7.6 Evaluation Progression

Run	Prompts	max_tokens	rep_penalty	min_tokens	full_ft	lora_ft
Run 1	100	256	1.2	50	53%	54%
Run 2	100	256	1.5	50	56%	46%
Run 3	100	256	1.5	15	52%	54%
Run 4	100	512	1.5	15	61%	55%
Run 5	500	512	1.5	15	56.6%	52.2%

Increasing the generation limit from 256 to 512 tokens produced the largest improvement.

7.7 Cosine Metric Limitations

The cosine similarity metric introduces several biases.

Bias	Effect
Short reference bias	Concise answers score higher
Vocabulary overlap	Synonyms reduce similarity score
Length penalty	Longer answers penalised
Repetition inflation	Repeated phrases inflate similarity

For this reason, cosine similarity should be interpreted as a secondary metric.

7.8 Qualitative Case Studies

This section highlights several representative prompt comparisons between the base model and the fine-tuned model.

Editing Task

Prompt:

Edit this sentence to make it sound more natural.

Base model output drifts into unrelated text.

The fine-tuned model preserves the meaning while improving fluency.

Factual Question

Prompt:

Determine the number of weeks in a year.

Both models answer correctly, but the fine-tuned model produces a clearer and more concise explanation.

Structured Planning

Prompt:

Develop a list of five steps to ensure project success.

The fine-tuned model produces a structured numbered list aligned with the prompt requirements.

7.9 Formal Success Criteria

The project success condition was defined as achieving a win-rate greater than 55% against the base model.

Model	Target Metric	Achieved Result	Status
full_ft	$\geq 55\%$ vs base	61.0%	Pass
lora_ft	$\geq 55\%$ vs base	55.0%	Pass
qlora_ft	$\geq 55\%$ vs base	55.0%	Pass

All three fine-tuned models successfully meet the project objective.

8 Diagnostic Findings and Fixes Applied

During development, several diagnostic investigations were conducted to identify the causes of early evaluation failures.

The following table summarises key findings and their resolutions.

Finding	Severity	Root Cause	Fix Applied
Repetition loops	High	<code>min_tokens</code> forcing generation past EOS	Repetition penalty increased; n-gram truncation added
LoRA scale error	High	scale=1.0 underpowered adapters	Enforced scale=2.0
Terse training data	Medium	Many Alpaca answers under 30 words	Filtered dataset to longer responses
System prompt mismatch	Medium	Inference injected system prompts	Aligned prompt format
Short context training	Medium	Early configs used 256 tokens	Increased to 512 tokens
Undertraining	Medium	Validation loss still decreasing	Extended training to 2 epochs
No overfitting observed	Positive	Small train/val gap	No action required
Model selection verified	Verification	Best trial correctly selected	Validated via config inspection
No data leakage	Positive	Train/eval separation verified	No action required

Several additional runtime issues were also addressed during development.

8.1 Post-Development Fixes

Issue	Root Cause	Resolution
ArrayAt error in MLX	Incompatible array assignment API	Rewrote logits processors using list conversion
Slow reasoning judge	Large reasoning models	Added streaming and early-exit parsing
Evaluation progress loss	Buffered writes	Switched to incremental append with auto-resume
Slow evaluation loops	HTTP overhead	Added local MLX judge option

9 Complete Changeable Parameter Reference

This section lists the main parameters that can be modified throughout the pipeline.

9.1 Dataset Parameters

Parameter	Current Value	Effect
DATASET_NAME	tatsu-lab/alpaca	Source dataset
NUM_EXAMPLES	5000	Size of filtered dataset
TRAIN_SPLIT	0.8	Training fraction
VALID_SPLIT	0.1	Validation fraction
TEST_SPLIT	0.1	Test fraction
MIN_ANSWER_WORDS	30	Minimum response length

9.2 LoRA Parameters

Parameter	Baseline	Best Sweep	Range
rank	16	32	4–64
scale	2.0	2.0	1.0–4.0
dropout	0.05	0.05	0–0.3
lora_layers	16	16	1–22
keys	q,v	q,k,v,o	projection layers

9.3 Training Parameters

Parameter	Baseline	Best Sweep	Range
learning_rate	$1e^{-5}$	$5e^{-5}$	$1e^{-6}$ – $1e^{-3}$
batch_size	1	1	1–8
grad_accumulation	1	16	1–64
iters	1200	8000	variable
max_seq_length	512	512	128–2048

9.4 Evaluation Parameters

Parameter	Value	Purpose
temperature	0.2	sampling randomness
top_p	0.9	nucleus sampling
max_tokens	512	output length limit
seed	42	reproducibility
min_tokens	15	prevents early termination
repetition_penalty	1.5	repetition control

10 Infrastructure and Environment

10.1 Hardware

Component	Requirement
CPU/GPU	Apple Silicon (M1/M2/M3/M4)
Memory	8GB minimum, 16GB recommended
Disk space	~15GB

10.2 Software Stack

All dependencies are pinned in `requirements.txt` at the repository root. Installation via:

```
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
```

Package & Category & Purpose
mlx & Core ML & Apple ML framework
mlx-lm & Core ML & LLM training and inference
datasets & Core ML & HuggingFace dataset loader
streamlit & UI & Interactive demo interface
fastapi / uvicorn & UI & REST API backend (legacy app)
sentence-transformers & Evaluation & Embedding-based cosine similarity
nltk & Evaluation & Text preprocessing
numpy & Evaluation & Numerical operations
requests & Evaluation & LM Studio API access
tabulate & Evaluation & Report formatting
matplotlib & Visualisation & Loss curve plotting
optuna & Optional & Hyperparameter search (commented out)

10.3 Makefile Commands

The project includes a Makefile that provides one-command access to all pipeline stages.

```
make install
```

```

make prepare
make eda
make validate
make train-lora
make train-qlora
make train-full
make sweep
make eval
make eval-lmstudio
make perplexity
make demo

```

A full reproduction workflow is:

```

make install
make prepare
make validate
make sweep
make eval
make demo

```

11 File Reference

11.1 Core Pipeline Files

File & Purpose
Makefile & Pipeline automation (one-command entry points)
requirements.txt & Pinned Python dependencies
prepare_dataset.py & Dataset curation (filter, split, format)
smart_train.py & Training wrapper with real-time early stopping
sweep_finetune.py & Grid / TPE hyperparameter sweep runner
validate_training_setup.py & Pre-run sanity checks
plot_loss.py & ASCII loss curve plotter from train.log
quick_eval.py & Base vs adapter side-by-side quick check
eda.py & Dataset EDA report + leakage/dedup checks
lora_config.yaml & Baseline LoRA configuration
qlora_config.yaml & Baseline QLoRA configuration

11.2 Evaluation Files

File & Purpose
evaluation/pipeline.py & Core evaluation library
evaluation/run_pipeline.py & Full pipeline one-command runner
evaluation/run_eval_6models.sh & 6-model shell wrapper
evaluation/generate_responses.py & Response generation for all models
evaluation/make_pairs.py & Blind pairwise comparison builder
evaluation/judge_with_lmstudio.py & LM Studio judge (streaming, SSE)
evaluation/judge_with_model.py & Local MLX fast judge
evaluation/score_judgments.py & Win-rate and CI computation
evaluation/score_similarity_rankings.py & Cosine similarity scoring
evaluation/check_response_quality.py & Gibberish / quality heuristics
evaluation/models.jsonl & 6-model evaluation spec
evaluation/eval_prompts.jsonl & 500-prompt frozen evaluation set
evaluation/eval_prompts_smoke20.jsonl & 20-prompt smoke subset
evaluation/eval_references.jsonl & Reference answers for cosine eval

11.3 Generated Artifacts

Directory & Contents
data/ & train.jsonl (4k), valid.jsonl (500), test.jsonl (500)
adapters/ & Baseline LoRA adapter checkpoints
models/ & tinyllama-4bit-base, phi-2-hf-4bit-mlx, qwen1.5-1.8b-chat-4bit
mlx_best_models/{full,lora,qlora}/ & Best adapter per technique from sweep
mlx_sweep_runs/{full,lora,qlora}/ & Trial configs, best.json, results.json
evaluation/runs/ & Timestamped eval outputs (responses, judgments, reports)

12 Reproducibility and Documentation

12.1 Repository

The complete source code, configuration files, and documentation are publicly available at:

<https://github.com/ysingh-grid/TinyLlamaFinetuning>

The repository contains:

- All training scripts, sweep configuration, and evaluation pipeline
- A `requirements.txt` listing all Python dependencies
- A `Makefile` providing reproducible one-command entry points
- A comprehensive `README.md` with step-by-step reproduction instructions
- An `evaluation/README.md` covering evaluation-specific setup and usage

12.2 Dependency Pinning

All dependencies are declared in `requirements.txt`:

```
# Core ML
mlx-lm
mlx
datasets

# UIs
streamlit
fastapi
uvicorn[standard]
pydantic

# Evaluation
sentence-transformers
nltk
numpy
requests
tabulate

# Visualization / EDA
matplotlib

# Optional: HPO
# optuna
```

Installation is handled by `make install`, which creates a `.venv` virtual environment and installs from `requirements.txt`.

Note: Always use `.venv/bin/python` rather than the system `python3` to ensure the correct MLX environment is active.

12.3 Step-by-Step Reproduction

Full end-to-end reproduction from a clean checkout:

```
# 1. Install dependencies into a virtual environment
make install

# 2. Prepare and filter the Alpaca dataset
make prepare          # produces data/train.jsonl, valid.jsonl, test.jsonl

# 3. Run dataset EDA and leakage checks
make eda              # produces data/eda_report.md

# 4. Validate the training setup
```

```

make validate

# 5. Run the hyperparameter sweep (or individual techniques)
make sweep          # all three techniques
# or individually:
make train-lora
make train-qlora
make train-full

# 6. Run 6-model pairwise evaluation
make eval           # local MLX fast judge (~8 min)
make eval-lmstudio  # LM Studio reasoning judge (higher quality)

# 7. Launch the Streamlit demo
make demo           # opens http://localhost:8501

```

Alternatively, individual scripts can be invoked directly:

```

# Data preparation
.venv/bin/python prepare_dataset.py

# Train best LoRA adapter
.venv/bin/python smart_train.py \
  --config mlx_sweep_runs/lora/trial_0002/config.yaml \
  --patience 5 --min-delta 0.0

# Evaluate (fast local MLX judge, 100 prompts)
JUDGE_MODE=model MAX_PROMPTS=100 ./evaluation/run_eval_6models.sh

# Evaluate (LM Studio judge)
JUDGE_MODE=lmstudio LMSTUDIO_MODEL=qwen/qwen3-4b \
  MAX_PROMPTS=100 ./evaluation/run_eval_6models.sh

```

12.4 Loss Curve Visualisation

Validation loss is logged to `{adapter_path}/train.log` at every evaluation step by the early stopping wrapper.

The repository includes `plot_loss.py`, a utility that reads this log file and produces an ASCII loss curve:

```

.venv/bin/python plot_loss.py \
  --log ./adapters/tinyllama-lora-alpaca/train.log

```

This provides a direct visual record of training convergence without requiring any additional tooling.

12.5 Quick Sanity Check

A lightweight evaluation script is provided to verify that a trained adapter improves on the base model without running the full evaluation pipeline:

```
.venv/bin/python quick_eval.py \  
  --adapter ./adapters/tinyllama-lora-alpaca \  
  --n-prompts 10
```

12.6 Fixed Seeds

Component & Seed

Dataset splitting & Fixed (constant in `prepare_dataset.py`)

Evaluation prompts & 42 (`eval_prompts.jsonl` frozen)

Generation & 42 (seed=42 passed to all models)

Pair ordering & Randomised, but deterministic given prompt seed

12.7 Troubleshooting

Issue & Resolution

`ModuleNotFoundError` & Use `.venv/bin/python`, not `system python3`

Out-of-memory on training & Reduce `batch_size` in config YAML

Missing similarity packages & `pip install sentence-transformers nltk numpy`

`ArrayAt` object error & MLX version mismatch; fixed in `pipeline.py`

Slow LM Studio judge & Switch to `JUDGE_MODE=model` for local MLX judge

Looping / repetition in output & Check `repetition_penalty=1.5` and `min_tokens=15`

Wrong model loaded & Verify model paths in `evaluation/models.json`
